软件过程与管理课程作业 Assgt_01

项目名称： 智能在线编程练习平台 (Smart OJ Platform)

1. 软件过程定义的来源

本项目软件过程的定义主要来源于以下三个维度，遵循 ISO/IEC 12207 关于过程定义的指导原则：

- **国际标准框架 (ISO/IEC 12207)：**
  过程定义的基础源自 ISO/IEC 12207 软件生命周期标准。我们参考了其中的核心流程，特别是**开发过程**（涵盖需求分析、设计、编码、测试）和**管理过程**（涵盖项目管理、配置管理）。标准中对“软件生存周期过程”的分类（主过程、支持过程、组织过程）构成了我们项目骨架的基石。

- **组织过程资产与工具链：**
  结合团队现有的技术栈和协作习惯，我们定义了具体的支撑过程。根据《立项计划书》，我们采用了 **GitHub** 进行版本控制（配置管理过程），使用 **飞书/语雀** 进行文档协作（基础设施过程）。技术选型上，基于 **SpringBoot** 的后端规范和 **Docker** 沙箱标准也构成了技术过程的来源。

- **项目特定需求与约束：**
  本项目具有“传统 OJ 判题 + AI 大模型”的双引擎特性，且受限于 12 周的短周期（2026.04 - 2026.06）。因此，过程定义必须来源于对**高风险技术点**（如代码沙箱安全、AI 接口集成）的特殊考量，这迫使我们在标准流程中增加了技术预研环节。

2. 项目软件过程的定义

结合 ISO 12207 标准与项目实际，我将本项目的软件过程定义为 **“基于时间盒的阶段性开发模型”**。考虑到项目周期仅为 12 周，且当前时间为 **2026 年 4 月 26 日**（即项目启动的第一周），我们正处于**定义阶段**向**构建阶段**过渡的关键期。

本项目的全生命周期过程定义如下表所示：

阶段	名称	核心活动 (Activities)	关键产出物 (Work Products)	对应 ISO 12207 流程
Phase 1	**定义与立项** (Week 1) *(当前阶段)*	1. 需求调研与可行性分析 2. 立项答辩与预算审批 3. 团队组建与角色分配 4. 制定初步 WBS (工作	• 项目立项计划书 • 预算规划表 • 风险识别清单	获取、供应、项目管理

阶段	名称	核心活动 (Activities)	关键产出物 (Work Products)	对应 ISO 12207 流程
		分解结构)		
Phase 2	**分析与设计** (Week 2-3)	1. 需求规格化 (SRS) 2. 系统架构设计 (SDD) 3. **技术 POC 验证** (沙箱/AI) 4. 数据库建模	• 软件需求规格说明书 • 软件设计说明书 • 核心架构 POC 报告	开发 (系统需求/架构)
Phase 3	**实现与构建** (Week 4-8)	1. 环境搭建与框架搭建 2. 模块化编码 (User/OJ/AI) 3. 单元测试与代码评审 4. 持续集成 (CI)	• 可运行源代码 • 单元测试报告 • API 接口文档	开发 (详细设计/编码)
Phase 4	**测试与交付** (Week 9-12)	1. 集成与系统测试 2. OJ 专项压力测试 3. AI 内容合规审计 4. 部署上线与验收	• 集成测试报告 • 用户手册 • 验收报告	开发 (测试)、运行

3. 软件过程剪裁的想法 (Tailoring Rationale)

根据 ISO/IEC 12207 关于过程剪裁的指导，我对标准过程进行了以下剪裁：

1. **裁剪了“正式的同行评审 (Formal Peer Review)”流程：**
    - **剪裁方案：** 移除了标准中繁琐的 Fagan 检查会议流程。
    - **理由：** 学生团队沟通成本低，且时间紧迫。改为采用 **GitHub Pull Request (PR)** 机制进行轻量级的代码评审。这既保证了代码质量（特别是安全沙箱代码），又符合现代开发习惯，提高了效率。
2. **合并了“详细设计”与“编码”阶段：**
    - **剪裁方案：** 不强制产出独立的《详细设计说明书》，而是将设计决策直接体现在代码注释和《系统设计说明书 (SDD)》的模块描述中。
    - **理由：** 项目采用敏捷思想，且使用 SpringBoot 等成熟框架，设计与实现高度耦合。过度的设计文档会产生维护成本，我们遵循“够用即可”的原则，将精力集中在核心逻辑（如判题机逻辑）的实现上。

3. **增加了“技术预研 (Spike) 子流程”：**
    - **剪裁方案：** 在 Phase 2（设计阶段）强制插入 3-5 天的 Spike 时间。
    - **理由：** 这是本项目最大的风险点。标准过程往往假设技术是成熟的，但本项目涉及 **Docker 沙箱逃逸防护**和 **AI 大模型接口调用**。如果不提前验证，后期重构成本极高。因此，剪裁了部分文档编写时间，用于优先验证 OJ 引擎和 AI 接口的可行性。
4. **简化了文档交付物集：**
    - **剪裁方案：** 仅保留核心的立项书、需求书、设计书、测试报告和用户手册。裁剪了标准中要求的《质量保证计划》、《验证计划》等独立计划文档。
    - **理由：** 项目性质为毕业设计/课程作业，非商业级大型软件。将质量保证和验证活动直接融入到每日站会和测试用例中，而非通过独立的文档流程来管理，以减少形式主义工作量。